

Map Auto-Filtering & Report Processing Architecture

Complete Technical Specification • File-by-File & Function-by-Function Flow

System Architecture Overview

The CoastalEye / JalDrishti platform provides an intelligent, dual-tiered coastal and urban water hazard management system. It features dynamic map auto-geolocation, strict cascading administrative filtering (State → City → Locality), multimodal AI/ML hazard verification (MobileNetV2 + Google Gemini), duplicate suppression (perceptual hashing), and automated priority ranking for municipal disaster response.

1. Map Automatic Filtering & Jurisdiction Engine

The real-time live map automatically handles geolocation auto-detection, cascading boundary filtering, and road-level zoom focusing. The architecture divides responsibilities across frontend geographic helpers, UI map components, and backend aggregation routes.

Module / Layer	File Path	Primary Functions & Role
Citizen Live Map	frontend/src/pages/Citizen/LiveMap.tsx	Cascading state/city filters, proximity sorting (calculateDistanceKm), URL param auto-focus, road-level zoom 17.
Government Map	frontend/src/pages/Government/LiveMap.tsx	Officer GPS auto-detection (handleAutoDetectLocation), administrative jurisdiction centering, live report drawer.
Map Viewport & UI	frontend/src/components/map/LiveMap.tsx	Leaflet map container, animated camera transitions (MapViewport / map.flyTo), severity color markers.
Location Dataset	frontend/src/data/indiaLocations.ts	36 Indian States/UTs, normalized cities (normalizeCityName), nearest jurisdiction finder (detectNearestJurisdiction).
Geolocation Hook	frontend/src/hooks/useGeolocation.ts	Wraps HTML5 navigator.geolocation with live accuracy and error boundaries.
Map Service Layer	frontend/src/services/mapService.ts	Executes HTTP GET /reports/map with parameterized administrative filters.
Backend Map API	Backend/app/reports/routes.py	Exposes GET /reports/map, GET /reports/nearby, and GET /reports/hotspots.
Backend Service	Backend/app/reports/service.py	Implements getMapReports(), getAdministrativeReports(), and spatial getHotspots().

Detailed Map Filtering & Auto-Centering Mechanics:

- Automatic GPS & Nearest Jurisdiction Detection:** In Government/LiveMap.tsx, handleAutoDetectLocation() retrieves the officer's live GPS coordinates and passes them to detectNearestJurisdiction(lat, lng) in indiaLocations.ts. The function computes the closest Indian administrative district via the Haversine formula and automatically selects the state and city, centering the map at Zoom 12.
- Cascading Sequential Filter (State → City → Locality):** In Citizen/LiveMap.tsx, selecting a State dynamically populates availableCities (STATE_CITIES_MAP merged with active database reports). Selecting a City populates availableLocalities (CITY_LOCALITIES_MAP). Names are standardized via normalizeCityName() (e.g., 'Kanpur Nagar' / 'Kanpur Dehat' → 'Kanpur').
- Dynamic Viewport Auto-Centering & Zoom Levels:** The activeViewport controller computes camera focus based on user interaction:
 - **Locality Level:** Deep road/street-level **Zoom 17**.
 - **City Level:** Municipality-wide **Zoom 13**.
 - **State Level:** Regional **Zoom 8**.
 - **Default GPS:** User device location at **Zoom 14**.Transitions are smoothly animated via MapViewport using Leaflet's map.flyTo([lat, lng], zoom).
- URL Query Deep-Linking:** Citizen/LiveMap.tsx listens for URL parameters (?lat=...&lng=...&zoom=...&locality=...&city=...&state=...). When opened from an alert or dashboard card, it automatically configures all filter states and flies directly to the specific incident pin.
- Live Device Proximity Sorting:** calculateDistanceKm() calculates the distance in kilometers from the citizen's live GPS coordinates to each hazard marker. Reports are dynamically reordered so the closest hazards appear at the top with formatted distance labels (e.g., '120m away' or '2.4 km away').
- Backend Map Filtering Pipeline:** GET /reports/map invokes getMapReports() in reports/service.py, querying MongoDB using case-insensitive regex on administrative fields and computing real-time incident clusters via getHotspots().

2. End-to-End Hazard Report Processing Pipeline

When a citizen submits a hazard report, the request travels across the frontend UI, client services, FastAPI router, file handlers, duplicate validators, MobileNetV2 ML service, Google Gemini AI, reverse geocoders, and priority rankers before persisting into MongoDB Atlas.

Step 1: Media Selection & Pre-Analysis AI Assist
Files: <code>frontend/src/pages/Citizen/ReportHazard.tsx</code> & <code>frontend/src/services/reportService.ts</code> Function: <code>handleMediaSelect(e)</code> in <code>ReportHazard.tsx</code> creates a local preview object URL. Function: <code>analyzeWaterMedia(file, categoryHint, captionHint)</code> in <code>reportService.ts</code> packages image into <code>multipart/form-data</code> and calls <code>POST /reports/analyze</code>. Backend Handler: <code>analyzeReportMedia()</code> in <code>Backend/app/reports/routes.py</code> validates file size (<10MB) and MIME type. Runs ML model first via <code>getOwnModelPrediction()</code>. If confidence < 0.70 or description is missing, triggers <code>analyzeHazardWithGemini()</code> in <code>gemini/service.py</code> to suggest title, description, and hazard type. The UI displays an AI assist suggestion card with an 'Apply AI Suggestion' button to auto-fill the form.
Step 2: Form Submission & Draft Assembly
Files: <code>frontend/src/pages/Citizen/ReportHazard.tsx</code> & <code>frontend/src/services/reportService.ts</code> Function: <code>handleSubmit(e)</code> in <code>ReportHazard.tsx</code> validates required fields: <code>problemType</code> and <code>effectiveLocation</code>. Constructs <code>SubmitReportDraft</code> object (title, <code>problemType</code>, description, coords, address, mediaFiles, severity, contact info). Function: <code>submitWaterReport(draft)</code> in <code>reportService.ts</code> appends all fields to <code>FormData</code> (image, latitude, longitude, <code>claimedHazard</code>, locality, city, district, state). Executes <code>HTTP POST /reports/</code> via <code>Axios</code> instance in <code>api.ts</code>.
Step 3: Backend Controller & File Persistence
Files: <code>Backend/app/reports/routes.py</code> & <code>Backend/app/utils/fileHandler.py</code> Route Controller: <code>addReport()</code> in <code>Backend/app/reports/routes.py</code> receives multipart form fields and image. Extracts authenticated user from token dependency <code>get_optional_user</code> (<code>userId</code>, <code>username</code>). Function: <code>saveImage(image, 'uploads/reports')</code> in <code>fileHandler.py</code> generates a UUID-based filename and writes image bytes to disk.
Step 4: Image Perceptual Hash & Duplicate Detection
Files: <code>Backend/app/reports/validation.py</code> & <code>Backend/app/utils/imageUtils.py</code> Function: <code>validateDuplicateReport(imagePath, lat, lng, hazardType, userId)</code> in <code>validation.py</code> generates perceptual image hash using <code>get_image_hash()</code>. Queries active database reports from the past 48 hours within a 150-meter radius (<code>DUPLICATE_RADIUS_METERS</code>). Calculates image hash similarity via <code>hash_similarity()</code>: • Exact Duplicate (≥ 95% similarity): Returns <code>{ isDuplicate: true, duplicateType: 'exact', existingReportId }</code> preventing duplicate spam. • Potential Duplicate: Nearby report within 150m with identical hazard type logged for corroboration.
Step 5: Machine Learning Classification (Microservice)
Files: <code>Backend/app/ml_client.py</code> & <code>ml-service/app/routers/detect.py</code> Function: <code>getOwnModelPrediction(imagePath)</code> in <code>Backend/app/ml_client.py</code> makes an asynchronous <code>HTTP POST</code> request with the image file to <code>http://localhost:8001/api/detect</code>. ML Endpoint: <code>detect_hazard(file)</code> in <code>ml-service/app/routers/detect.py</code> calls <code>classify_image(image_bytes)</code> in <code>ml-service/app/models/image_model.py</code>. Executes MobileNetV2 inference and returns predicted <code>hazard_type</code>, confidence, and base severity.
Step 6: Conditional Google Gemini AI Verification
Files: <code>Backend/app/gemini/service.py</code> & <code>Backend/app/gemini/prompt.py</code> Trigger Condition: Executed if ML confidence is below 0.70, ML result is unknown/normal, or citizen title/description are empty. Function: <code>analyzeHazardWithGemini()</code> in <code>gemini/service.py</code> optimizes and downscales image to 384×384 JPEG and calls Gemini Multimodal Vision API via <code>_call_gemini_vision_rest()</code>. Enforces quality gate: rejects irrelevant photos (selfies/indoor) and auto-generates realistic incident title and situational description.

Step 7: Reverse Geocoding & Jurisdiction Enforcement
Files: Backend/app/utils/geocode.py & Backend/app/reports/routes.py Function: reverseGeocode(latitude, longitude) in Backend/app/utils/geocode.py queries OpenStreetMap Nominatim API (zoom=18&addressdetails;=1) to resolve state, district, city, suburb, road, and formatted address. In routes.py, landmark keywords (e.g. 'PSIT', 'Green Park', 'VIP Road') enforce precise Indian municipal jurisdictions (e.g., Kanpur Nagar, Uttar Pradesh).
Step 8: Proximity Corroboration & Dynamic Priority Scoring
Files: Backend/app/reports/validation.py & Backend/app/models/report_ranker.py Function: checkNearbyReports(lat, lng, category, radius_km=1.0) in validation.py counts active neighborhood reports for incident density. Function: calculate_priority_score() in report_ranker.py computes multi-factor score (0-100): Score = 0.25×Severity + 0.15×People + 0.15×Hazard + 0.10×Recency + 0.10×Confidence + 0.15×Density + 0.10×Reliability Function: get_priority_level(score) maps score to: CRITICAL (≥80), HIGH (≥60), MEDIUM (≥40), or LOW (<40).
Step 9: Database Persistence & Cluster Assignment
Files: Backend/app/reports/service.py & Backend/app/database.py Function: createReport(reportData) in Backend/app/reports/service.py generates unique public report tracking ID via generate_public_report_id() (e.g., JAL-2026-B812F4). Assigns clusterId by checking nearby spatial proximity or generating a new cluster identifier. Initializes report audit timeline with initial event: { status: 'submitted', title: 'Report Submitted' }. Executes database.reports.insert_one(reportDocument) in MongoDB Atlas.
Step 10: Response Return & Frontend UI Real-Time Update
Files: Backend/app/reports/routes.py & frontend/src/pages/Citizen/ReportHazard.tsx Backend returns JSON with { success: true, reportId: 'JAL-2026-XXXX', report: {...} }. In ReportHazard.tsx, handleSubmit receives the response, triggers a success toast notification, and displays the confirmation modal. Citizen can copy the Public Report ID to clipboard with one click or navigate directly to /citizen/track/:id.

3. Complete Technical Matrix (File vs. Function vs. Action)

Reference table summarizing all files, functions, protocols, and actions involved across both workflows:

Layer	File Path	Function / Symbol	Protocol / Method	Action / Purpose
Frontend UI	Citizen/LiveMap.tsx	hazards, activeViewport	React useMemo	Cascading filters & road-level zoom 17 auto-centering.
Frontend UI	Government/LiveMap.tsx	handleAutoDetectLocation	HTML5 Geolocation	Auto-detects officer's administrative jurisdiction.
Frontend UI	ReportHazard.tsx	handleSubmit	React Form Event	Constructs draft payload and triggers report creation.
Frontend Data	indiaLocations.ts	detectNearestJurisdiction	Haversine Distance	Calculates closest Indian city from live coordinates.
Frontend Service	reportService.ts	analyzeWaterMedia	POST /reports/analyze	Sends image to backend for initial AI analysis.
Frontend Service	reportService.ts	submitWaterReport	POST /reports/	Submits multipart report payload to FastAPI backend.
Backend Route	reports/routes.py	addReport	FastAPI POST Endpoint	Coordinates verification, ML, Gemini, geocoding, and DB save.
Backend Validation	reports/validation.py	validateDuplicateReport	pHash + Haversine (150m)	Suppresses identical images (≥95% match) & checks duplicates.
Backend Client	ml_client.py	getOwnModelPrediction	HTTP POST /api/detect	Sends image to Python MobileNetV2 ML service.
ML Microservice	ml-service/app/main.py	classify_image	MobileNetV2 Inference	Predicts water hazard class, confidence, and severity.
Backend AI	gemini/service.py	analyzeHazardWithGemini	Google Gemini REST	Multimodal verification, quality gate, and dynamic narrative.
Backend Geocode	utils/geocode.py	reverseGeocode	Nominatim OSM API	Resolves coordinates into structured locality, city, state.
Backend Ranking	report_ranker.py	calculate_priority_score	7-Factor Weighted Algorithm	Calculates priority score (0-100) and severity level.
Backend Persistence	reports/service.py	createReport	MongoDB insert_one	Assigns JAL-2026-XXXX ID, cluster ID, and audit timeline.